52

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Mesaj Kuyruğu (Distributed Message Queue / Kafka Klonu) Tasarımı

Designing a Distributed Message Queue (Kafka Clone): Zero-Copy & Commit Log

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ KAFKA KLONU PERFORMANS SIRRI: ZERO-COPY DISK-TO-NETWORK ]

GELENEKSEL YÖNTEM (4 KULLANICI ALANI - KERNEL KOPYASI, YAVAŞ!):
Disk ──1. DMA──► OS Cache ──2. CPU──► JVM Heap ──3. CPU──► Socket Buffer ──4. DMA──► NIC

KAFKA ZERO-COPY YÖNTEMİ (LINUX sendfile(): SIFIR JVM KOPYASI!):
Disk ──1. DMA Kopya──► OS PageCache ──2. DMA Kopya──► Network Interface Card (NIC)
(CPU ve JVM tamamen atlanır; diskten doğrudan ağ kartına ışık hızında akar!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normalde bir bilgisayarın diskindeki bir dosyayı internete göndermek, bir kargocunun paketi depodan alıp kendi ofis masasına getirmesi, paketi açması, tekrar bantlaması ve sonra kamyona taşıması gibidir (4 kez gereksiz yere taşınır). Kafka'nın Sıfır Kopyalama (Zero-Copy) sihri ise kargocuyu aradan çıkarıp depodan çıkan raylı sistemi doğrudan kamyonun kasasına bağlamaktır! Veri asla işletim sisteminin veya Java'nın hafızasına kopyalanıp vakit kaybetmez; doğrudan diskten ağ kablosuna fırlar!

🇬🇧 ENGLISH (ELI5)

Traditional file transmission over the network is like a warehouse worker loading a box onto their office desk, unboxing it, inspecting it, taping it shut again, and then carrying it to the delivery truck (4 redundant data copies and context switches!). Kafka's Zero-Copy optimization connects a direct gravity conveyor chute from the warehouse shelf straight into the back of the cargo truck! Data never touches JVM user-space memory; the Linux kernel routes raw disk PageCache bytes directly onto the Network Interface Card (NIC) at hardware line speed!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Dağıtık Mesaj Kuyruğu Tasarımının 3 Performans Direği: 1) Sıralı Disk Erişimi (Sequential IO): Diskler rastgele okumada (Random Seek) yavaştır ancak sıralı yazmada (Sequential Append) SSD'lerde 2-3 GB/sn hızlara ulaşır (RAM hızına çok yakındır). Kafka tüm mesajları sadece kütüğün sonuna ekler (Append-only). 2) Linux sendfile() (Zero-Copy): `transferTo()` sistem çağrısıyla veriler OS PageCache'ten doğrudan NIC buffer'ına kopyalanır. Context Switch (Kullanıcı alanı - Çekirdek geçişi) sayısı 4'ten 2'ye, veri kopyası sayısı 4'ten 2'ye düşer; CPU tüketimi sıfırlanır. 3) Toplu İşleme (Batching & Compression): Mesajlar tek tek değil, 64 KB'lık paketler halinde LZ4/ZSTD ile sıkıştırılarak ağdan aktarılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The 3 engineering pillars of Kafka's throughput mastery: 1) Sequential Disk IO: Mechanical and NVMe storage drives deliver massive bandwidth (gigabytes/sec) when written sequentially, rivaling raw DRAM speeds. Kafka leverages append-only commit logs exclusively. 2) Linux Zero-Copy (`sendfile` system call): Traditional reads transfer bytes: Disk -> PageCache -> User Space -> Socket Buffer -> NIC (4 buffer copies, 4 context switches). The `sendfile()` primitive transfers bytes directly from PageCache to the Network Interface Card via DMA, executing zero user-space memory copies and zero CPU load. 3) Batch Compression: Producers aggregate micro-messages into 64KB batches, compressing with LZ4/Snappy before transmission, maximizing network MTU utilization.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Zero-Copy (FileChannel transferTo) Simülasyonu
import java.nio.channels.*;

public class ZeroCopyDemonstrator {
    // Java NIO FileChannel.transferTo() arka planda Linux sendfile() kullanır!
    public static void zeroCopyTransfer(FileChannel sourceFile, WritableByteChannel targetSocket)
throws Exception {
        long position = 0;
        long count = sourceFile.size();

        // Veri JVM User-Space belleğine HİÇ girmeden doğrudan diskten ağ soketine uçar!
        while (position < count) {
            long transferred = sourceFile.transferTo(position, count - position, targetSocket);
            position += transferred;
        }
        IO.println("✔ Zero-Copy Transferi Tamamlandı! Boyut: " + count + " bytes");
    }
}

void main() {
    IO.println("Zero-Copy Metodu: FileChannel.transferTo() Hazır");
    IO.println("Avantaj: Sıfır JVM Garbage Collection, Sıfır CPU Buffer Kopyası!");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Mesajları hızlı olsun diye RAM'de (JVM Heap) saklarım' demeyin! JVM Heap'te milyonlarca mesaj saklamak Garbage Collection (GC) duraklamalarına yol açar ve sunucuyu kilitler. Kafka gibi sistemler işletim sisteminin PageCache'ini kullanarak disk ve RAM'i birlikte en verimli şekilde kullanır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose holding high-volume message queues within the JVM heap. Millions of heap objects trigger catastrophic Stop-The-World garbage collection pauses. Real systems delegate caching entirely to the underlying OS PageCache.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Sıralı disk yazması (Sequential IO) rastgele bellek erişiminden bile daha hızlı olabilir.
- Linux sendfile() Zero-Copy tekniği CPU ve JVM hafıza kopyalama yükünü tamamen sıfırlar.
- Toplu mesajlaşma (Batching) ağ başlık yükünü minimize ederek throughput'u katlar.

🎯 Key Takeaways for the Interview

- Sequential disk writes achieve multi-gigabyte throughput, outpacing random memory access.
- Linux sendfile() zero-copy streams bytes directly from disk cache to the network card.
- Batching micro-messages into compressed frames maximizes network packet density.